

ZEBA ACADEMY / zeba-academy / Instructor: Gufyar bin Uzayr
Lab Practicum 01: Empirical Parallel Computing
Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

Duration: 90 Minutes

Format: Solo In-Class Lab

Weight: 5% of Course Grade

STUDENT NAME: Ruslan

STUDENT ID: 230103026

HOST CPU / SOC MODEL: AMD Ryzen 7 7435HS (Zen 3+)

OPERATING SYSTEM & KERNEL: Windows 11 Pro for Workstations (Kernel 10.0.26200)

Generative AI & Tooling Policy: Authorized to use LLMs for code generation. All execution timings, memory counters, and hardware cache metrics must be physically benchmarked on the host machine.

TASK 1: Host Silicon Audit & Flynn's Taxonomy Mapping

15 Minutes | 20 Points

Query your operating system and CPU registers to document physical core topology, cache boundaries, and vector math extensions.

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	AMD Ryzen 7 7435HS (x86_64)	Base & Max Boost Clock	3.10 GHz / 4.50 GHz
Physical Hardware Cores	8 Physical Cores	Logical Cores (SMT/Threads)	16 Hardware Threads
L1 Data / Instruction Cache	L1d: 256 KB L1i: 256 KB	L2 Cache (Per Core / Total)	4096 KB Total (512 KB/Core)
L3 Shared Last-Level Cache	16 MB Shared LLC	SIMD Vector Extensions	AVX, AVX2, FMA3, SSE4.2, AES

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

- SISD (Single Instruction, Single Data):** Sequential single-threaded execution (e.g. single-threaded compiler pass or single-core script). One CPU core executes a single instruction stream on one data stream.
- SIMD (Single Instruction, Multiple Data):** Vector processing via AVX2/FMA3 256-bit vector registers. A single SIMD instruction simultaneously operates on parallel vector data elements (e.g. matrix/array math).
- MIMD (Multiple Instruction, Multiple Data):** Multi-core parallel execution (e.g. OpenMP, Python multiprocessing pool). Independent physical cores execute distinct instruction streams on separate memory spaces asynchronously.

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall

35 Minutes | 30 Points

Workload Algorithm: Prime Sieve (0 to 3,000,000) | Language & Runtime: Python 3.12 multiprocessing (Process Pool)

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time T_N (s)	Speedup $S_N = T_1/T_N$	Efficiency $E_N = S_N/N$
N = 1 (Baseline)	8.0730	7.9589	7.7813	7.9377	1.00×	100.0%
N = 2	5.0750	5.3597	5.2461	5.2270	1.52×	75.9%
N = 4	3.1598	3.3389	3.5153	3.3380	2.38×	59.5%
N = 8	2.4705	2.3819	2.1788	2.3437	3.39×	42.3%
N = 16	1.9948	1.8698	1.8712	1.9120	4.15×	25.9%
N = 32	2.0159	2.0574	2.0923	2.0552	3.86×	12.1%

Q2.1: Scaling Saturation & Hardware Contention Analysis:

- Performance peaks at **N = 16 threads ($S_{16} = 4.15\times$)** and reverses at **N = 32 threads ($S_{32} = 3.86\times$)**.
- Physical vs. Logical Cores:** Strong scaling occurs from N=1 to N=8 (3.39× speedup) as each worker maps to a physical core. From N=8 to N=16, SMT hardware threads share execution resources per core, yielding diminished marginal scaling.
 - Contention Wall (N=32):** When N=32 exceeds total hardware threads (16), OS thread scheduling overhead (context switching latency), L3 cache line thrashing, and RAM bus contention degrade total throughput.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap**25 Minutes | 25 Points**

Spawn 10 parallel processes/threads. Each executes 1,000,000 increments on a shared global integer. Theoretical expected total: 10,000,000.

Run #	Measured Output	Error (10 ⁷ - Act)	Run #	Measured Output	Error (10 ⁷ - Act)
#1	2,021,744	7,978,256	#6	1,789,374	8,210,626
#2	1,851,422	8,148,578	#7	1,808,836	8,191,164
#3	1,800,709	8,199,291	#8	1,704,467	8,295,533
#4	1,804,383	8,195,617	#9	1,759,310	8,240,690
#5	1,863,943	8,136,057	#10	1,798,261	8,201,739

Q3.1: Read-Modify-Write (RMW) Window Deconstruction:

Incrementing shared memory requires 3 assembly operations: **1) LOAD** value from RAM/L3 into CPU register, **2) ADD** 1 in ALU, **3) STORE** updated register back to memory. Without atomicity, Core A and Core B concurrently read the same memory value (e.g. 500). Both compute 501 and write it back, resulting in a Lost Update hazard where 1 increment is completely overwritten.

Q3.2: The Synchronization Penalty:

Execution time: **Unlocked = 616.80 ms** vs. **Locked (Mutex) = 72,138.26 ms** (~117× slowdown).

Why synchronized code is substantially slower: Acquiring/releasing a Mutex 10,000,000 times forces serialized execution across cores, generating heavy inter-core bus locking, MESI cache coherency line invalidation traffic, and kernel context switching overhead.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon**15 Minutes | 15 Points****1. Parallel Fraction Derivation (p) Formula: $p = 2 \times (T_1 - T_2) / T_1$**

$T_1 = 7.9377 \text{ s}$ | $T_2 = 5.2270 \text{ s}$ → **Calculated p = 0.6830** (0.682994)

2. Amdahl Theoretical Speedup Ceiling: $S_{\max} = 1 / (1 - p)$

Sequential fraction (1 - p) = **0.3170** | **Max Speedup ($N \rightarrow \infty$) = 3.15×**

3. Amdahl 64-Core Projection: $S_{64} = 1 / [(1 - p) + (p / 64)]$

Projected Speedup on 64 Cores = 3.05×

4. Gustafson Scaled Speedup: $S_{\text{Gustafson}} = (1 - p) + (p \times 64)$

Projected Scaled Speedup (64× Problem) = 44.03×

Q4.1: Strong vs. Weak Scaling Analysis:

- Amdahl's Law (Strong Scaling):** Fixes workload size. As cores $N \rightarrow \infty$, parallel time shrinks to zero, making the sequential overhead (1 - p = 31.70%) an absolute speedup bottleneck (capped at 3.15× regardless of core count).
- Gustafson's Law (Weak Scaling):** Scales workload size proportionally with core count ($N=64$). Because parallel work increases 64-fold while sequential overhead remains constant, parallel speedup approaches near-linear scaling (44.03×).

```
PS C:\Users\rusla> python C:\Users\rusla\Downloads\benchmark_task2.py
=== TASK 2: MULTI-THREAD SCALING BENCHMARK ===
N= 1 | Run 1: 8.0730 s
N= 1 | Run 2: 7.9589 s
N= 1 | Run 3: 7.7813 s
--> N= 1 Avg Time: 7.9377 s

N= 2 | Run 1: 5.0750 s
N= 2 | Run 2: 5.3597 s
N= 2 | Run 3: 5.2461 s
--> N= 2 Avg Time: 5.2270 s

N= 4 | Run 1: 3.1598 s
N= 4 | Run 2: 3.3389 s
N= 4 | Run 3: 3.5153 s
--> N= 4 Avg Time: 3.3380 s

N= 8 | Run 1: 2.4705 s
N= 8 | Run 2: 2.3819 s
N= 8 | Run 3: 2.1788 s
--> N= 8 Avg Time: 2.3437 s

N=16 | Run 1: 1.9948 s
N=16 | Run 2: 1.8698 s
N=16 | Run 3: 1.8712 s
--> N=16 Avg Time: 1.9120 s

N=32 | Run 1: 2.0159 s
N=32 | Run 2: 2.0574 s
N=32 | Run 3: 2.0923 s
--> N=32 Avg Time: 2.0552 s

--- FINAL SUMMARY TABLE ---
N  Run 1 (s)  Run 2 (s)  Run 3 (s)  Avg TN (s)  Speedup SN  Efficiency
1  8.0730    7.9589    7.7813    7.9377      1.00      x100.0
2  5.0750    5.3597    5.2461    5.2270      1.52      x75.9
4  3.1598    3.3389    3.5153    3.3380      2.38      x59.5
8  2.4705    2.3819    2.1788    2.3437      3.39      x42.3
16 1.9948    1.8698    1.8712    1.9120      4.15      x25.9
32 2.0159    2.0574    2.0923    2.0552      3.86      x12.1
```

